



DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Laboratorio de datos

Web Scraping

Primer cuatrimestre 2025

Mateo Guerrero Schmidt

```
pip install bs4
```

Introducción a Web Scraping

Es una técnica utilizada para extraer información de páginas web. Especialmente útil cuando los datos no están estructurados, o sea, no están en archivos .csv u otros.

¿Cómo podemos acceder a una página?

Para hacerlo desde el código tenemos que hacer un *request* al servidor.

Haciendo request con Python

Desde python usamos la librería nativa 'requests'

```
import requests

url = "https://lcd.exactas.uba.ar/materias/"
r = requests.get(url)
```



Plan de estudios

La función get nos devuelve toda la información de la página

****Importante**** cada vez que ejecutamos el comando get estamos enviando una consulta al servidor. Hay que ser responsables en el uso de las consultas ya que si se ejecutan demasiadas pueden generar problemas en el servidor o incluso pueden bloquear nuestra IP por uso indebido.

Status Code

Los códigos de estado nos indican si el pedido al servidor fue exitoso

- 2__: asociado a una solicitud exitosa
- 4__: asociado a un problema en el cliente (ej. Error 404)
- 5__: asociado a un problema del servidor

```
print(r.status_code)
```

200

```
url2 = "https://lcd.exactas.uba.ar/materiassss/"  
print(requests.get(url2).status_code)
```

404

¿Cómo accedemos al contenido?

```
r.text
```

```
'<!DOCTYPE html>\n<html class="no-overflow-y avada-html-layout-wide avada-html-header-position-top avada-is-100-percent-template avada-header-color-not-opaque" lang="es-AR" prefix="og: http://ogp.me/ns# fb: http://ogp.me/ns/fb#">\n<head>\n\t<meta http-equiv="X-UA-Compatible" content="IE=edge" /\n\t<meta http-equiv="Content-Type" content="text/html; charset=utf-8"/>\n\t<meta name="viewport" content="width=device-width, initial-scale=1" /\n\t<title>Plan de estudios &#8211; Licenciatura en Datos &#8211; Exactas &#8211; UBA</title>\n\t<style>\r\n\t\t#wpadminbar #wp-admin-bar-cp_plugins_top_button .ab-icon:before {\r\n\t\t\tcontent: "\\f533";\r\n\t\t\ttop: 3px;\r\n\t\t}\r\n\t\t#wpadminbar #wp-admin-bar-cp_plugins_top_button .ab-icon {\r\n\t\t\ttransform: rotate(45deg);\r\n\t\t}\r\n\t</style>\r\n\t<style>\n#wpadminbar #wp-admin-bar-vtrts_free_top_button .ab-icon:before {\n\t\tcontent: "\\f185";\n\t\tcolor: "#1d4523";\n\t\ttop: 3px;\n\t}\n</style>\n<meta name="d1m version'
```

```
print(len(r.text))
```

```
122184
```

r.text nos devuelve todo la página en formato HTML pero en un solo string

Parsers

Los parsers son analizadores sintácticos. Es decir, traducen el string gigante en un objeto más cómodo de trabajar.

En python usamos la librería 'BeautifulSoup'

```
from bs4 import BeautifulSoup
materias = BeautifulSoup(r.text, 'html.parser')
materias
```

```
<!DOCTYPE html>
```

```
<html class="no-overflow-y avada-html-layout-wide avada-html-header-position-top
avada-is-100-percent-template avada-header-color-not-opaque" lang="es-AR"
prefix="og: http://ogp.me/ns# fb: http://ogp.me/ns/fb#">
<head>
<meta content="IE=edge" http-equiv="X-UA-Compatible"/>
<meta content="text/html; charset=utf-8" http-equiv="Content-Type"/>
<meta content="width=device-width, initial-scale=1" name="viewport"/>
<title>Plan de estudios – Licenciatura en Datos – Exactas – UBA</title>
```

HTML

Es un lenguaje para crear estructuras de páginas web.

Los **elementos** son la estructura más básica de los HTML.

Estos elementos contienen **etiquetas** (tags) que le dan propiedades. Algunos ejemplos:

- `<div>`: elemento general
- `<p>`: para escribir texto en formato de párrafos
- `<h1>`, `<h2>`, `<h3>`: usado para títulos y subtítulos
- `<a>`: para hipervínculos
- `<img>`: para insertar imágenes
- `<ol>`, `<ul>`: para definir listas ordenadas (ordered list) y listas no ordenadas (unordered list)
- `<li>`: para definir los elementos de las listas

HTML - atributos de elementos

Cada **elemento**, a su vez, admite **atributos**. Algunos dependen de la **etiqueta**.

Por ejemplo:

- *width*: define el ancho de un elemento
- *id*: indica un código único para un elemento
- *style*: un diccionario con las características visuales del elemento
- *href*: indica la url para un hipervínculo
- *src*: indica la ubicación de una imagen

HTML - Clases

Las **clases** son un **atributo** de los elementos.

Permiten agrupar elementos dentro de un mismo tipo para asignarles un formato visual similar.

Ojo! **No** confundir con las **etiquetas**.

Ejemplo de código

Facultad de Ciencias Exactas y Naturales - UBA

La Facultad de Ciencias Exactas y Naturales de la UBA ofrece diversas carreras en el área de la ciencia y la tecnología.

Carreras disponibles

- Licenciatura en Ciencias de la Computación
- Licenciatura en Matemática
- Licenciatura en Ciencia de Datos
- Licenciatura en Física
- Licenciatura en Ciencias Biológicas

Para más información, visita la [página oficial de Exactas UBA](https://exactas.uba.ar).

Metadatos
No visibles

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Facultad de Ciencias Exactas y Naturales - UBA</title>
</head>
```

```
<body>
```

Cuerpo
Todo lo visible

```
  <h1>Facultad de Ciencias Exactas y Naturales - UBA</h1>
  <p>La Facultad de Ciencias Exactas y Naturales de la UBA ofrece diversas carreras en el área de la ciencia y la tecnología.</p>
  <h2>Carreras disponibles</h2>
  <ul>
    <li id="computacion" class="carrera destacada">Licenciatura en Ciencias de la Computación</li>
    <li id="matematica" class="carrera">Licenciatura en Matemática</li>
    <li id="datos" class="carrera">Licenciatura en Ciencia de Datos</li>
    <li id="fisica" class="carrera">Licenciatura en Física</li>
    <li id="biologia" class="carrera">Licenciatura en Ciencias Biológicas</li>
  </ul>
  <p>Para más información, visita la <a href="https://exactas.uba.ar">página oficial de Exactas UBA</a>.</p>
</body>
</html>
```

```
r = requests.get("https://lcd.exactas.uba.ar/materias/")  
materias = BeautifulSoup(r.text, 'html.parser')
```

¿Como extraemos información?

La función **find()** de BeautifulSoup nos devuelve el primer elemento que encuentra con la etiqueta pedida (la etiqueta es opcional):

```
materias.find('h1')
```

```
<h1 class="entry-title">Plan de estudios</h1>
```

La función **find_all()** devuelve una lista con todos los elementos con esa etiqueta:

```
materias.find_all('h1')
```

```
[<h1 class="entry-title">Plan de estudios</h1>,  
 <h1 class="title-heading-center" style="margin:0;color:#2981a2;"><b>Plan de  
<span style="color: #9bd279;">estudios</span></b></h1>]
```

```
r = requests.get("https://lcd.exactas.uba.ar/materias/")
materias = BeautifulSoup(r.text, 'html.parser')
```

¿Como extraemos información?

A las funciones **find()** y **find_all()** les podemos pedir que, además de la etiqueta, cumpla con algunos atributos:

```
materias.find_all('li', attrs={'style':True})
```

```
[<li style="font-weight: 400;"><span style="font-weight: 400;">Modelos de
regresión</span></li>,
 <li style="font-weight: 400;"><span style="font-weight: 400;">Herramientas de
visualización de datos</span></li>,
 <li style="font-weight: 400;"><span style="font-weight: 400;">Estimación no
paramétrica aplicada</span></li>,
 <li style="font-weight: 400;"><span style="font-weight: 400;">Aprendizaje
```

```
materias.find_all('h3', attrs={'class':'flip-box-heading-back'})
```

```
[<h3 class="flip-box-heading-back" style="color:#ffffff;">Datos</h3>,
 <h3 class="flip-box-heading-back" style="color:#ffffff;">Investigación operativa y
Optimización</h3>,
 <h3 class="flip-box-heading-back" style="color:#ffffff;">Estadística matemática-
computacional</h3>,
 <h3 class="flip-box-heading-back" style="color:#ffffff;">Modelado continuo</h3>
```

```
r = requests.get("https://lcd.exactas.uba.ar/materias/")
materias = BeautifulSoup(r.text, 'html.parser')
```

¿Como extraemos información?

Podemos acceder a los hijos de un elemento usando **find()** y **find_all()**:

```
div = materias.find('ul')
div.find_all('li')
```

```
[<li class="menu-item menu-item-type-post_type menu-item-object-page menu-item-home menu-
item-2009" data-item-id="2009" id="menu-item-2009"><a class="fusion-textcolor-highlight"
href="https://lcd.exactas.uba.ar/"><span class="menu-text">Inicio</span></a></li>,
 <li class="menu-item menu-item-type-custom menu-item-object-custom menu-item-has-children
menu-item-2010 fusion-droddown-menu" data-item-id="2010" id="menu-item-2010"><a
```

La función **get()** devuelve el valor de un atributo para un elemento:

```
print(materias.find('h1').get('style'))
print(materias.find('h2').get('style'))
```

```
None
color:#f9f9fb;
```

```
materias.find('h1').get('class')
```

```
['entry-title']
```

```
r = requests.get("https://lcd.exactas.uba.ar/materias/")  
materias = BeautifulSoup(r.text, 'html.parser')
```

¿Como extraemos información?

Algunos elementos pueden tener múltiples clases:

```
div = materias.find(attrs={'class': 'fusion-layout-column'})  
div.get('class')
```

```
['fusion-layout-column',  
 'fusion_builder_column',  
 'fusion-builder-column-0',  
 'fusion_builder_column_1_1',  
 '1_1',  
 'fusion-flex-column']
```

Si se quiere buscar un elemento que cumpla con más de una clase a la vez se puede usar **select()**

¿Cómo podemos acceder a la información del cronograma sugerido?

Primero, busquemos un elemento distintivo que agrupe todos los datos que queremos.

Con **F12** podemos acceder al inspector de HTML.

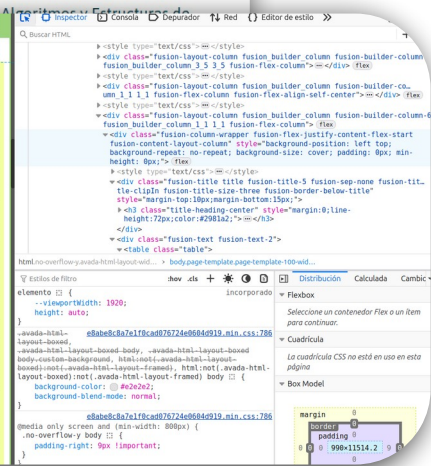
Nos permite ver la estructura y también los atributos de los elementos.

Nos ayuda a encontrar los elementos en el código

Cronograma sugerido

Cuatrimestre	Asignatura	Correlatividad de Asignaturas
3	Álgebra I	CBC
3	Algoritmos y Estructuras de Datos I	CBC
4	Análisis I	CBC

Cuatrimestre	Asignatura	Correlatividad de Asignaturas
3	Álgebra I	CBC
3	Algoritmos y Estructuras de Datos I	CBC
4	Análisis I	CBC
4	Algoritmos y Estructuras de Datos II	Algoritmos y Estructuras de Datos I - Álgebra I
5	Análisis II	Análisis I
5	Álgebra Lineal Computacional	Algoritmos y Estructuras de Datos II
5	Laboratorio de Datos	Algoritmos y Estructuras de Datos I
6	Análisis Avanzado	Algebra I - Algoritmos y Estructuras de Datos I
6	Electiva de Introducción a las Ciencias Naturales	CBC
7	Probabilidad	Análisis Avanzado
7	Algoritmos y Estructura de Datos III	Algoritmos y Estructuras de Datos II
	Intr. a la Estadística y Ciencia de Datos	Lab. de Datos, Probabilidad, Álgebra Lineal Computacional



¿Cómo podemos acceder a la información del cronograma sugerido?

Busquemos un elemento distintivo que agrupe todos los datos que queremos.

```
divs = materias.find_all(attrs={'class':'table'})  
len(divs)
```

1

Leamos todos los valores de la tabla

```
div = materias.find(attrs={'class':'table'})  
rows = div.find_all('tr') #tr: table row  
for row in rows:  
    celdas = row.find_all('td')  
    print(celdas[0].text, celdas[1].text, celdas[2].text)
```

Cuatrimestre Asignatura Correlatividad de Asignaturas

3 Álgebra I CBC

3 Algoritmos y Estructuras de Datos I CBC

4 Análisis I CBC

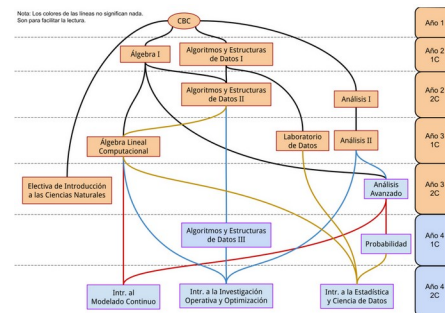
4 Algoritmos y Estructuras de Datos II Algoritmos y Estructuras de Datos I – Álgebra I

Extraer imágenes

```
materias.find_all('img')
```

```
[,  
  <img alt="Licenciatura en Datos – Exactas – UBA Logo" class="fusion-sticky-logo" data-  
retina_logo_url="https://lcd.exactas.uba.ar/wp-content/uploads/2023/06/lic-datos-
```

```
url_imagen = materias.find(attrs={'image_id':"2395"}).get('src')  
  
imagen = requests.get(url_imagen)  
  
with open("imagen.png", 'wb') as f:  
    f.write(imagen.content)
```



Algunas conclusiones

- El web scraping es un trabajo bastante artesanal.
- No todas las páginas van a tener la misma estructura.
- No siempre es fácil extraer los elementos que tenemos.
- Suele haber varias maneras de llegar a la información.

Ejercicio

Armar un DataFrame que para cada uno de los cursos propuestos para el último ciclo, guarde todas las materias correspondientes al mismo.

	Titulo	Materias
0	Datos	[Modelos de regresión, Herramientas de visuali...
1	Investigación operativa y Optimización	[Programación lineal entera avanzada, Programa...
2	Estadística matemática-computacional	[Modelo lineal, Herramientas de visualización ...
3	Modelado continuo	[Análisis numérico, Elementos de análisis fun

```
Camino = pd.DataFrame(columns=['Titulo', 'Materias'])

camino = materias.find_all(attrs={'class':'fusion-flip-box-wrapper'})
for camino in caminos:
    titulo = camino.find('h3').text
    listado = camino.find_all('li')

    cursos = []
    for curso in listado:
        cursos.append(curso.text)

    Camino.loc[len(Camino)] = [titulo, cursos]
Camino
```

Mi solución:

Todo esto, ¿es legal?

El web scraping es una práctica legal, pero los sitios web pueden imponer algunas restricciones.

El archivo **robots.txt** actúa como una guía para los bots y scrapers indicando qué secciones del sitio pueden ser accedidas.

En sí, robots.txt es una sugerencia y su cumplimiento no es obligatorio, pero de todos modos es altamente recomendable respetarlo. Ignorarlo puede generar consecuencias como el bloqueo de nuestra IP.

Además, es recomendable revisar los términos y condiciones de cada página para conocer su política respecto a la extracción de datos. Algunas plataformas prohíben explícitamente el scraping en sus términos de uso, y no respetarlo podría derivar en acciones legales o sanciones.

robots.txt

El archivo robots.txt separa las restricciones para los distintos tipo de usuarios. Para esto, hace uso del comando **User-agent**. En caso de querer hacer referencia a todos los se indica con un asterisco.

Luego, se usan las palabras Disallow y Allow para indicar qué secciones están habilitadas para acceder.

Dos comandos comunes suelen ser '*Disallow: /*' que indica que se bloquea la totalidad de las secciones; y '*Disallow:* ' que indica que no se bloquea ninguna de las secciones.

```
url = "https://exactas.uba.ar/robots.txt"
r = requests.get(url)
robots_exactas = BeautifulSoup(r.text, 'html.parser')
robots_exactas
```

```
User-agent: *
Disallow: /calendar/action-posterboard/
Disallow: /calendar/action-agenda/
Disallow: /calendar/action-oneday/
Disallow: /calendar/action-month/
Disallow: /calendar/action-week/
Disallow: /calendar/action-stream/
Disallow: /calendar/action-undefined/
Disallow: /calendar/action-http/
Disallow: /calendar/action-default/
Disallow: /calendar/action-poster/
Disallow: /calendar/action-*/
Disallow: /*controller=ailec_exporter_controller*
Disallow: /*action-*/
```

Wikipedia robots.txt

```
url = "https://wikipedia.org/robots.txt"
r = requests.get(url)
robots_wikipedia = BeautifulSoup(r.text, 'html.parser')
robots_wikipedia

# Observed spamming large amounts of https://en.wikipedia.org/?curid=NNNNNN
# and ignoring 429 ratelimit responses, claims to respect robots:
# http://mj12bot.com/
User-agent: MJ12bot
Disallow: /

# advertising-related bots:
User-agent: Mediapartners-Google*
Disallow: /

# Wikipedia work bots:
User-agent: IsraBot
Disallow:

User-agent: Orthogaffe
Disallow:
```

APIs

APIs

Algunas páginas nos facilitan el acceso a la información a través de APIs (Application Programming Interface). Estas son interfaces que nos permiten interactuar con una página web o un sistema de manera estructurada y eficiente.

Las APIs funcionan como puentes de comunicación entre diferentes aplicaciones, permitiendo que un programa solicite y reciba datos sin necesidad de acceder directamente al código fuente del otro.

La información suele entregarse en formatos estandarizados como JSON (JavaScript Object Notation) o XML (eXtensible Markup Language), lo que facilita su procesamiento en distintos lenguajes de programación.

Archivos .JSON

Problemos con la **pokeapi**

```
url_poke = "https://pokeapi.co/api/v2/pokemon/charizard"
```

```
r_poke = requests.get(url_poke)  
r_poke.status_code
```

```
200
```

```
pokemon = r_poke.json()  
pokemon.keys()
```

```
dict_keys(['abilities', 'base_experience', 'cries', 'forms', 'game_indices',  
'height', 'held_items', 'id', 'is_default', 'location_area_encounters', 'moves',  
'name', 'order', 'past_abilities', 'past_types', 'species', 'sprites', 'stats',  
'types', 'weight'])
```

```
pokemon['types']
```

```
[{'slot': 1,  
  'type': {'name': 'fire', 'url': 'https://pokeapi.co/api/v2/type/10/'}},  
{ 'slot': 2,  
  'type': {'name': 'flying', 'url': 'https://pokeapi.co/api/v2/type/3/'}}]
```

Usamos la URL para acceder a la info que queremos

El parser para archivos .JSON nativo de python

Los archivos JSON funcionan como diccionarios anidados

Ejercicio

Comparar la relación de cambio de distintas monedas a partir de la api de **CoinGecko** ("https://api.coingecko.com/api/v3/exchange_rates").

Por ejemplo, comparar la relación de cambio del peso argentino con el bitcoin, el peso chileno, y el yen japonés.

Pista:

```
url = "https://api.coingecko.com/api/v3/exchange_rates"  
data = requests.get(url).json()  
data
```

API wrappers

Un API Wrapper es una librería o módulo que actúa como una capa de abstracción sobre una API. Su propósito es facilitar el acceso a la API sin que el usuario tenga que hacer llamadas HTTP manuales.

Miremos el módulo de python para la API de Wikipedia

```
!pip install Wikipedia-API  
import wikipediaapi
```

```
wiki = wikipediaapi.Wikipedia(user_agent='Clase-APIs (mateogs01@gmail.com)', language='es')
```

La API de Wikipedia exige un user_agent para poder controlar los malos usos.

Más info en: https://foundation.wikimedia.org/wiki/Policy:Wikimedia_Foundation_User-Agent_Policy

API wrappers

```
arg = wiki.page('Argentina')

print(f"Título: {arg.title}")

print("\nResumen:\n")
print(arg.summary)

print("\nSecciones principales:")
for seccion in arg.sections:
    print(f"- {seccion.title}")
```

Título: Argentina

Resumen:

Argentina, oficialmente República Argentina,[a] es un país soberano de América del Sur, ubicado en el extremo sur y sudeste de e Se constituye como un país federal descentralizado, compuesto por veintitrés provincias más la Ciudad Autónoma de Buenos Aires, La Constitución Nacional Argentina rige los principios de adhesión como una república representativa con veintitrés estados asoc Argentina se reconoce como un país bicontinental, cuyo vasto territorio abarca gran parte del Cono Sur y se extiende hasta la An Es el segundo país con el mayor índice de desarrollo humano (IDH) de la región, detrás de su vecino Chile.[12][13]Garantiza mode La República Argentina es una de las naciones más desarrolladas e influyentes del continente americano. Hasta mediados del siglo Con un desarrollo científico y tecnológico referente, es el país latinoamericano más laureado con premios Nobel, con cinco en to Asimismo, ha tenido personalidades significativas a lo largo de la historia, con contribuciones destacadas en deportes, ciencias Argentina integra el G20 –bloque que reúne a las naciones más ricas e industrializadas del planeta– y es miembro fundador del Me Su territorio bicontinental abarca una superficie de 2 780 400 km²,[3] es el país hispanohablante más extenso del planeta, el se Su territorio reúne una gran diversidad de climas. causada por una amplitud latitudinal que supera los 30° –incluyendo varias zo

Foto del día de la NASA

```
url = "https://api.nasa.gov/planetary/apod?api_key=DEMO_KEY"  
data = requests.get(url).json()  
print(f"Imagen del día: {data['title']} \n{data['url']}")
```

Imagen del día: Milky Way Through Otago Spires

https://apod.nasa.gov/apod/image/2507/MwSpires_Chay_960.jpg

